

Mean-Variance Optimization & the Efficient Frontier

Don't pick assets — pick the combination. Building the efficient frontier across six asset classes, and the two portfolios everyone quotes.

Harry Markowitz's 1952 insight launched modern portfolio theory: don't pick assets in isolation, pick the *combination* that gives the most return per unit of risk. We build the **efficient frontier** from real data across six asset classes and find the two portfolios everyone quotes — **minimum variance** and **maximum Sharpe**.

PROJECT CARD

CATEGORY

Portfolio Optimization

LIBRARIES

NumPy · Pandas · PyPortfolioOpt · Matplotlib

ASSETS

SPY, TLT, GLD, VNQ, VEA, VWO

TIMEFRAME

Jan 2015 – Dec 2024

USE

Strategic asset allocation — the benchmark every other method is measured against

1. Summary

Mean-variance optimization treats a portfolio's expected return as the weighted average of its assets, and its risk as the *portfolio* variance — which depends not just on each asset's volatility but on how they **co-move**. Minimizing variance for each level of target return traces out the **efficient**

frontier: the set of portfolios you can't beat. Two points on it get special names — the **minimum-variance** portfolio (leftmost) and the **maximum-Sharpe** (tangency) portfolio, where the risk-adjusted return peaks.

2. Intuition

2.1 The only free lunch in finance

Combine two assets that don't move in lockstep and the portfolio's risk is *less* than the weighted average of their risks — because their wiggles partly cancel. That cancellation is diversification, and it's why a stock-bond-gold mix can have lower volatility than any single sleeve.

2.2 Correlation does the heavy lifting

Volatility tells you how much one asset moves; **correlation** tells you whether they move together. Two 15%-vol assets

with -0.3 correlation build a far smoother portfolio than two with $+0.9$. The whole power of MVO lives in the covariance matrix.

2.3 What the frontier shows

Plot every possible portfolio in risk-return space and they fill a bullet-shaped cloud. Only the **upper-left edge** matters — for any risk level, that's the highest return available. Anything below it is a portfolio you'd never rationally hold.

3. Theory & Mechanics

With weights w , expected returns μ and covariance matrix Σ :

$$\text{portfolio return} = w^T \mu, \quad \text{portfolio variance} = w^T \Sigma w$$

The efficient frontier solves, for each target return μ^* :

$$\min_w w^T \Sigma w \quad \text{s.t.} \quad w^T \mu = \mu^*, \quad \sum_i w_i = 1$$

Maximum Sharpe maximizes $(w^T \mu - r_f) / \sqrt{w^T \Sigma w}$; **minimum variance** just minimizes $w^T \Sigma w$. We'll build the frontier two ways — a transparent NumPy Monte-Carlo cloud, then the exact optimum with **PyPortfolioOpt**.

4. Applied Example — Six ETFs

4.1 A diversified multi-asset universe

Six asset classes chosen to *not* move together — the raw material of a good frontier:

Ticker	Asset class
SPY	US equities (S&P 500)
TLT	Long US Treasuries
GLD	Gold
VNQ	US real estate (REITs)
VEA	Developed ex-US equities
VWO	Emerging-market equities

```

TICKERS = ["SPY", "TLT", "GLD", "VNQ", "VEA", "VWO"]
START, END = "2015-01-01", "2024-12-31"

def load_prices(tickers, start, end):
    """Adjusted-close prices: yfinance first, Stooq as fallback."""
    try:
        import yfinance as yf
        df = yf.download(tickers, start=start, end=end, auto_adjust=True, progress=False)["Close"]
        if not df.empty:
            return df[tickers].dropna()
    except Exception as exc:
        print(f"yfinance failed ({exc}); trying Stooq...")
    cols = {}
    for t in tickers:
        url = f"https://stooq.com/q/d/l/?s={t.lower()}.us&i=d"
        s = pd.read_csv(url, parse_dates=["Date"], index_col="Date")["Close"].rename(t)
        cols[t] = s
    return pd.concat(cols, axis=1).loc[start:end].dropna()

px = load_prices(TICKERS, START, END)
rets = px.pct_change().dropna()

mu = rets.mean() * 252          # annualised expected returns
Sigma = rets.cov() * 252       # annualised covariance
vol = np.sqrt(np.diag(Sigma))

summary = pd.DataFrame({"ann. return": mu, "ann. vol": vol,
                        "Sharpe": mu / vol}).round(3)
print(summary)
print(f"\n{len(px)} trading days, {px.index[0].date()} → {px.index[-1].date()}")

```

Ten years of daily data (2,515 trading days, 2015–2024). Note how different the standalone Sharpe ratios are — and that TLT earned essentially *nothing* over the decade:

Ticker	Ann. return	Ann. vol	Sharpe
SPY	13.9%	17.6%	0.79
TLT	−0.0%	15.3%	−0.00
GLD	8.5%	14.1%	0.60
VNQ	6.9%	20.8%	0.33
VEA	6.9%	17.3%	0.40
VWO	6.0%	19.8%	0.30

4.2 The correlation matrix — the source of the free lunch

Note where correlations are *low*: Treasuries (TLT, −0.21 to SPY) and gold (GLD, +0.05 to SPY) barely track equities, which is exactly why they smooth the portfolio.

```

corr = rets.corr()

fig, ax = plt.subplots(figsize=(6.5, 5.5))
im = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
ax.set_xticks(range(len(TICKERS))); ax.set_xticklabels(TICKERS)
ax.set_yticks(range(len(TICKERS))); ax.set_yticklabels(TICKERS)
for i in range(len(TICKERS)):
    for j in range(len(TICKERS)):
        ax.text(j, i, f"{corr.iloc[i,j]:.2f}", ha="center", va="center",

```

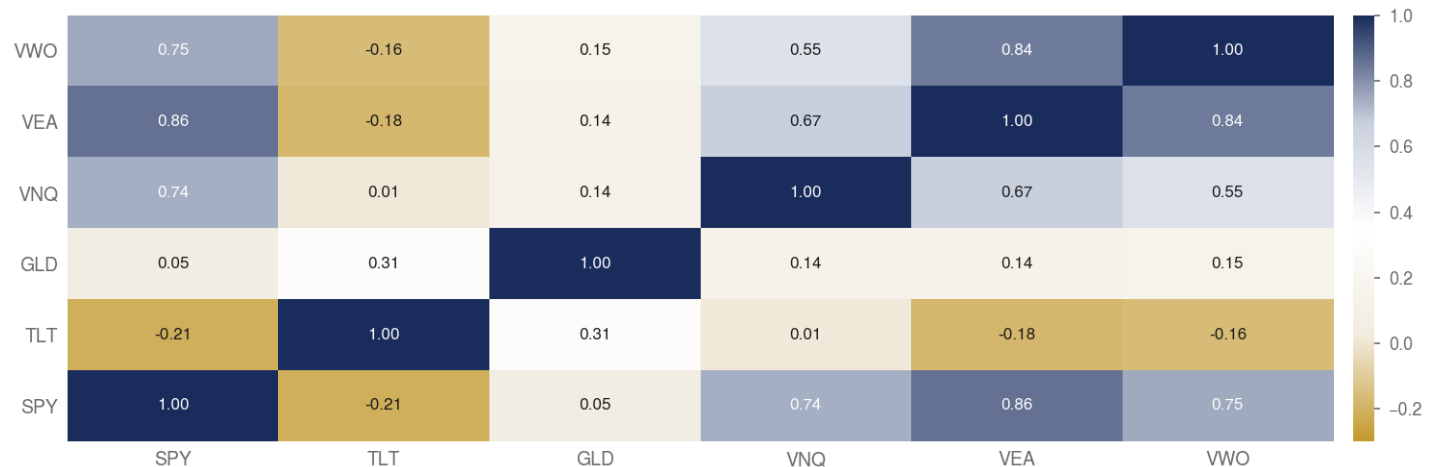
```

        color="white" if abs(corr.iloc[i,j]) > 0.5 else "black", fontsize=9)
fig.colorbar(im, fraction=0.046, pad=0.04)
ax.set_title("Correlation of daily returns")
plt.tight_layout(); plt.show()

```

Figure 4.2 · Correlation of daily returns, 2015–2024

The equity block (SPY, VNQ, VEA, VWO) runs 0.55 to 0.86; TLT is slightly negative against equities and GLD is near zero



Source: Author calculations, from the article's computed results.

4.3 The efficient frontier — Monte-Carlo cloud

Generate 20,000 random portfolios to see the bullet, then the frontier as its upper-left edge. Colour = Sharpe ratio.

```

N = 20_000
n = len(TICKERS)
rf = 0.02

w = np.random.dirichlet(np.ones(n), N)          # random long-only weights summing to 1
port_ret = w @ mu.values
port_vol = np.sqrt(np.einsum("ij,jk,ik->i", w, Sigma.values, w))
port_sharpe = (port_ret - rf) / port_vol

fig, ax = plt.subplots(figsize=(10, 6))
sc = ax.scatter(port_vol, port_ret, c=port_sharpe, cmap="viridis", s=6, alpha=0.5)
# individual assets
ax.scatter(vol, mu.values, c="red", marker="D", s=60, zorder=5)
for i, t in enumerate(TICKERS):
    ax.annotate(t, (vol[i], mu.values[i]), textcoords="offset points", xytext=(7, 3))
fig.colorbar(sc, label="Sharpe ratio")
ax.set_xlabel("Annualised volatility"); ax.set_ylabel("Annualised return")
ax.set_title(f"{N:,} random portfolios – the efficient frontier bullet")
plt.tight_layout(); plt.show()

```

4.4 The exact optima with PyPortfolioOpt

The cloud shows the shape; PyPortfolioOpt solves for the *exact* minimum-variance and maximum-Sharpe portfolios and overlays the true frontier.

```

from pypfport import EfficientFrontier, expected_returns, risk_models, plotting

mu_pp = expected_returns.mean_historical_return(px)
S_pp = risk_models.sample_cov(px)

```

```

# max Sharpe
ef = EfficientFrontier(mu_pp, S_pp)
ef.max_sharpe(risk_free_rate=rf)
w_sharpe = ef.clean_weights()
perf_sharpe = ef.portfolio_performance(risk_free_rate=rf)

# min variance
ef2 = EfficientFrontier(mu_pp, S_pp)
ef2.min_volatility()
w_minvar = ef2.clean_weights()
perf_minvar = ef2.portfolio_performance(risk_free_rate=rf)

weights = pd.DataFrame({"Max Sharpe": w_sharpe, "Min Variance": w_minvar}).round(3)
print(weights)
print(f"\nMax Sharpe : return {perf_sharpe[0]:.1%}, vol {perf_sharpe[1]:.1%}, Sharpe {perf_sharpe[2]:.2f}")
print(f"Min Variance: return {perf_minvar[0]:.1%}, vol {perf_minvar[1]:.1%}, Sharpe {perf_minvar[2]:.2f}")

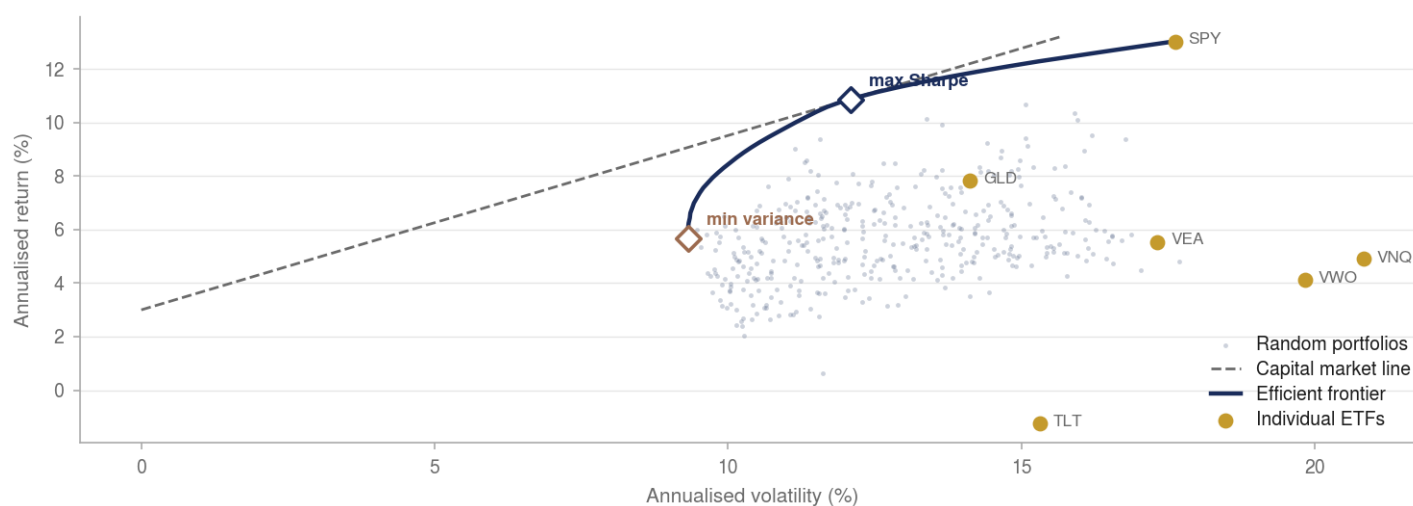
```

Ticker	Max Sharpe	Min Variance
SPY	0.563	0.268
TLT	0.000	0.371
GLD	0.437	0.280
VNQ	0.000	0.000
VEA	0.000	0.081
VWO	0.000	0.000

Max Sharpe: 10.8% return at 11.9% vol (Sharpe 0.73) — a two-asset SPY + GLD barbell. **Min variance:** 5.7% return at 9.3% vol (Sharpe 0.40) — note it holds 37% TLT *despite* TLT's zero return, purely for its negative correlation. That is the free lunch in action: the optimizer pays for co-movement, not for standalone performance.

Figure 4.4 · 20,000 random portfolios, the exact frontier, and the two special portfolios

The random portfolios form a bullet; the frontier runs along its upper-left edge, with the capital market line tangent at maximum Sharpe



Source: Author calculations, from the article's computed results. The cloud is drawn from the stored sample of the simulation.

Every single ETF plots *inside* the cloud, well below the frontier — even SPY, the decade's best performer, sits under the line. No individual asset is efficient; only combinations are.

5. Conclusion

5a. Strengths

- **Quantifies diversification** — turns “don’t put all your eggs in one basket” into an exact weight vector
- **One framework, any universe** — stocks, bonds, gold, real estate all go in the same optimizer
- **Closed-form and fast** — the frontier is a quadratic program that solves instantly
- **The foundation** — every allocation method (risk parity, Black-Litterman, CVaR) is a response to MVO

5b. Weaknesses & Limitations

- **Garbage in, garbage out** — expected returns are notoriously hard to estimate, and MVO is hypersensitive to them
- **Concentrated, unstable weights** — tiny input changes can swing allocations wildly (the “error-maximization” critique)
- **Backward-looking** — historical covariance assumes the past regime persists

- **Ignores tail risk** — variance treats upside and downside symmetrically; crashes aren’t Gaussian

5c. Applications in Practice

- Strategic asset allocation for pension funds and endowments
- Setting the neutral portfolio a discretionary manager tilts away from
- The benchmark every alternative allocation method is measured against

5d. Alternatives & Extensions

- **Black-Litterman** — fixes the input-sensitivity problem by blending market equilibrium with views (the next tutorial)
- **Risk parity** — sidesteps return estimation entirely by allocating on risk contribution (our futures-based piece)
- **Hierarchical Risk Parity** — uses clustering instead of matrix inversion for stabler weights
- **CVaR optimization** — replaces variance with a genuine tail-risk measure